DATABASE: everything you need for the DS, on one sheet

Scope: Chapter I (operators → normalization) + Chapter II (filtering → views)

Contents

PART A: Chapter I

- A1. Data types and basic commands
- A2. SQL operators
- A3. Relational model vocabulary
- A4. Keys
- A5. Integrity constraints
- A6. Relationships
- A7. ERD
- A8. Database schema
- A9. Normalization

PART B: Chapter II

- B1. Filtering
- B2. Joins
- B3. UNION
- B4. GROUP BY / HAVING
- B5. Subqueries
- B6. INSERT / UPDATE / DELETE / REPLACE
- B7. MySQL constraints
- B8. ALTER TABLE
- B9. Views

Then: a self-test, and the classic DS traps.

PART A: CHAPTER I

A1. Data types and basic commands (what you used in the labs)

Data	Type	Example
Whole number (ID, quantity)	`INT`	5
Number with decimals (price, salary)	`DECIMAL(p,s)` : p = total digits, s = digits after the point	`DECIMAL(6,2)` → 1500.50
Approximate decimal number	`FLOAT`	12.5
Text of **variable** length (name, email, address)	`VARCHAR(n)`	`VARCHAR(100)`
Text of **fixed** length ("always 9 characters")	`CHAR(n)`	`CHAR(9)` → Custom001
Date	`DATE`	'2026-03-25' (YYYY-MM-DD)
Date + time	`DATETIME`	'2026-03-25 14:30:00'

☐ A phone number goes in `VARCHAR`, not INT: it can start with 0 or contain +, and you never calculate with it.

```
CREATE DATABASE mz_devices;      -- create a database
  USE mz_devices;                  -- select it
  SHOW DATABASES;  SHOW TABLES;    -- list
  DESCRIBE devices;                -- structure of a table (columns + types)
  SELECT * FROM devices;           -- display all the content
  DROP TABLE address;              -- delete a table (structure + data)
  DROP DATABASE mz_devices;        -- delete a database
```

```
-- Lab 1 style: a DEFAULT value for the most common city
  CREATE TABLE address (
    CustomerID INT,
    Street VARCHAR(255),
    Postcode VARCHAR(10) DEFAULT 'HA97DE',
    Town VARCHAR(50) DEFAULT 'Harrow',
    PRIMARY KEY (CustomerID)
  );
```

AUTO_INCREMENT (from your `auto_increment.txt`): the DBMS generates the ID itself (1, 2, 3...). You don't give it in the INSERT.

```
CREATE TABLE orders (OrderID INT AUTO_INCREMENT PRIMARY KEY, Product VARCHAR(50));
  INSERT INTO orders (Product) VALUES ('Phone');     -- OrderID = 1 automatically
```

A2. SQL operators

Arithmetic works on numeric columns, in **SELECT** (to calculate) or **WHERE** (to filter):

Op	In SELECT	In WHERE
`+`	`SELECT salary + allowance FROM employee;`	`WHERE salary + allowance = 25000`
`-`	`SELECT salary - tax ...`	`WHERE salary - tax = 50000`
`*`	`SELECT tax * 2 ...`	`WHERE tax * 2 = 4000`
`/`	`SELECT allowance / salary * 100 ...` (a %)	`WHERE allowance / salary * 100 >= 5`
`%`	`SELECT hours % 2 ...` **(remainder)**	`WHERE hours % 2 = 0` → even

⚠ `%` = **modulus (remainder)**, not a percentage. 11 % 2 = 1, 10 % 2 = 0.
Comparison (used in WHERE): `=`, `<>` or `!=` (both mean "not equal"), `>`, `>=`, `<`, `<=`. Text needs **quotes**: `WHERE name = 'James'`. Numbers don't: `WHERE id = 1`.

A3. Relational model vocabulary

The relational model is built on **3 concepts: Data, Relationships, Constraints**. A database is "a collection of inter-related relations (tables)". **SQL** is the language used to query it.

Term	Meaning
Relation	a table
Tuple / record	a row
Attribute / field	a column
Domain	the set of acceptable values for a column (depends on its type)
Key	the attribute(s) that uniquely identify a row
Degree	the number of **columns**
Cardinality	the number of **rows** (records)

Example: Student(name, address, phone, email) with 100 students → **degree 4, cardinality 100**.

A4. Keys

Key	Definition	Example (Vehicle table, Lab 3)
Candidate key	any attribute whose values are unique and not null, so it could be the PK	VehicleID, PlateNumber, OwnerPhone
Primary key (PK)	the candidate key you chose: **unique + NOT NULL**, one per table	VehicleID
Alternate key	a candidate key that was **not** chosen as PK	PlateNumber, OwnerPhone
Composite key	a PK made of **2 or more columns**	(StudentID, CourseID)
Foreign key (FK)	a column that refers to the **PK of another table**	OwnerID in Vehicle → Owner(OwnerID)

Lab 3 trap: OwnerID is unique in the 3 sample rows, but it's **not** a candidate key, because one owner can have several vehicles. Judge the real meaning of the data, not just the sample rows. Also, a **name** is never a good key: two people can share a name.
How to choose the PK: short, unique, never null, never changes. An ID is better than a phone or plate number, which can change.

A5. Integrity constraints (the 3 conditions every relation must meet)

1. **Key constraint**: the key is **unique** and **cannot be NULL**. (Two students can't have the same StudentID.)
2. **Domain constraint**: the value must have a **valid data type** (or a valid range). Letters in a numeric column violate it.
3. **Referential integrity**: based on **foreign keys**. The FK value must **exist** as a PK in the referenced table. (`order_id` in Order_Item must exist in Order.)

A6. Relationships

Type	Meaning	Example	How it's implemented
1:1	each A ↔ exactly one B	Country ↔ Capital, Passenger ↔ Ticket	FK on either side + **UNIQUE**
1:N	one A → zero, one or many B; each B → one A	Customer → Orders, Parent → Children	**FK on the "many" side (N)**
M:N	many A ↔ many B	Students ↔ Courses, Players ↔ Games	**Junction table** with both FKs = composite PK

⚠ **M:N relationships are never kept in the data model.** They're broken into **2 one-to-many relationships** with a **junction (middle) table**.

A7. ERD (entity relationship diagram)

- An **entity** is a **box with 2 compartments**: the name on top, the attributes (with data types) below. Mark the **PK** and **FK**.
- **Lines** depend on cardinality:
  - **1:1** → a straight line
  - **1:N** → a line with a **crow's foot on one side** (the "many" side)
  - **M:N** → **crow's feet on both sides**

```
Department                  Course                        Student
  PK DepartmentNumber INT     PK CourseID INT               PK StudentID INT
     DepartmentName VARCHAR      CourseName VARCHAR            StudentName VARCHAR
     HeadOfDepartment VARCHAR    Credits INT                   DateOfBirth DATE
                                 FK DepartmentNumber INT       FK CourseID INT
          |1 ——————————< N|           |1 ——————————————< N|
```

(One department offers many courses; one course has many students.)
How to build an ERD in the DS:

1. Identify the entities (the nouns).
2. List the attributes and their types.
3. Choose a PK for each entity.
4. Find each relationship type from the sentences ("each … one or many …").
5. Put the FK on the N side, and create a junction table for each M:N.
6. Write the cardinality on each line.

A8. Database schema

A **schema** is the **blueprint** of the database. **Schema objects** are tables, columns and relationships at a minimum, plus data types, views, stored procedures, primary keys and foreign keys. A schema contains: all the important data and their relationships, unique keys for all entries, and a name and data type for every column.

- **Logical schema** = the **ERD** (entities, attributes, relationships).
- **Physical schema** = the **SQL code** (`CREATE DATABASE` + `CREATE TABLE` with types and keys).

The restaurant example (course):

- Customer 1:N Reservation
- Tbl 1:N Reservation
- Tbl 1:N Table_Order
- Waiter 1:N Table_Order
- Menu 1:N Menu_Item
- Table*Order* ***M:N*** *Menu*Item, through **Order*Menu*Item** (PK = order*id* + *menu*item_id, plus quantity)

```
CREATE TABLE order_menu_item(
    order_id INT, menu_item_id INT, quantity INT,
    PRIMARY KEY (order_id, menu_item_id),
    FOREIGN KEY (order_id) REFERENCES table_order(order_id),
    FOREIGN KEY (menu_item_id) REFERENCES menu_item(menu_item_id));
```

⚠ **Creation order:** create the referenced tables (those without FKs) **first**, then the tables that have FKs.

A9. Normalization

Aims: minimize data duplication, avoid errors during data modifications, simplify queries.

Form	Rule	How to fix it
Unnormalized	several values in one cell, repeating groups, no clear PK	–
1NF	**atomicity**: one single value per cell, plus no repeating groups	split into separate tables, one per entity
2NF	already 1NF + **no partial dependency**	move the attribute into its own table with the part of the key it depends on
3NF	already 2NF + **no transitive dependency**	move the dependent attributes into their own table

- **Partial dependency**: the table has a **composite PK**, and a non-key attribute depends on **only one part** of it.
- **Transitive dependency**: a non-key attribute depends on **another non-key attribute**.

Course example (Medical surgery):

1. **Unnormalized** table: Doctor, Region, Patients (several per cell), Surgery, Council, Postcode, Slot, Cost.
2. **1NF:**
   - Patient(PatientID, PatientName, SlotID, TotalCost), **PK (PatientID, SlotID)**
   - Doctor(DoctorID, DoctorName)
   - Surgery(SurgeryNumber, Region, Council, Postcode)
3. **2NF:** PatientName depends only on PatientID, and TotalCost depends only on SlotID. So:
   - Patient(PatientID, PatientName)
   - Appointment(**AppointmentID**, SlotID, TotalCost), with a new unique key
4. **3NF:** in Surgery, Postcode depends on Council (both non-key), which is transitive. So:
   - Location(SurgeryNumber, Postcode)
   - Council(Council, Region)
5. **Finally:** link all the tables with **foreign keys**.

How to answer a normalization question:

1. Say why it's unnormalized (multi-valued cells and repeating groups).
2. 1NF: one value per cell, one table per entity, give the PKs.
3. In each table with a **composite key**, ask: does an attribute depend on only part of the key? If so, split → 2NF.
4. Ask: does a non-key attribute determine another non-key attribute? If so, split → 3NF.

PART B: CHAPTER II

The examples below use these two tables:

Customers

CustomerID	FullName
1	Amal
2	Bilel
3	Chaima

Bookings

BookingID	BookingDate	Guests	CustomerID
10	2026-06-01	2	1
11	2026-06-01	4	1
12	2026-06-05	3	2
13	2026-06-09	5	NULL

Chaima (3) has **no booking**. Booking 13 has **no customer**.

B1. Filtering (WHERE), sorting, aliases

WHERE filters rows in **SELECT, UPDATE and DELETE**.

Need	Write
Both conditions true	`WHERE a AND b`
At least one true	`WHERE a OR b`
Opposite	`WHERE NOT Country = 'Mexico'`
In a list	`WHERE Country IN ('Germany','UK')` (= several ORs)
In a range	`WHERE Price BETWEEN 10 AND 20` (**includes** 10 and 20)
Text pattern	`WHERE Name LIKE 'a%'`
Missing value	`WHERE Phone IS NULL` (⚠ never `= NULL`)

LIKE: `%` = any number of characters (even 0), `_` = exactly 1 character.

Pattern	Meaning
`'a%'`	starts with a
`'%i'`	ends with i
`'%li%'`	contains "li"
`'_a%'`	a is the 2nd letter
`'a___%'`	starts with a, **at least 4** characters
`'a%i'`	starts with a and ends with i

Sorting: `ORDER BY col ASC` (the default) or `DESC`. `ORDER BY Dept, Salary DESC` sorts by Dept first, then by Salary. **Only some columns:** `SELECT FullName, Country FROM ...` (instead of `*`).
No duplicates: `SELECT DISTINCT City FROM Customers;`
Aliases (a temporary name):

- Column: `SELECT FullName AS Name`.
- With a space: `AS "Full Name"`.

- Table: `FROM Customers AS c` (then write `c.FullName`).
- Concatenate: `SELECT CONCAT(Address, ', ', City) AS Address FROM Customers;`
- **3 uses (knowledge check):** combine output into one column with concatenation, create distinct table names when using several tables, rename a name that's too long or technical.

B2. JOINS

A join links records from several tables using a **common column** (PK = FK). You need it when the information is spread over more than one table.

```
SELECT T1.col, T2.col
  FROM T1  <TYPE> JOIN  T2
  ON T1.common_col = T2.common_col;
```

INNER JOIN: only the rows that **match in both** tables.

```
SELECT Customers.FullName, Bookings.BookingID
  FROM Customers INNER JOIN Bookings ON Customers.CustomerID = Bookings.CustomerID;
```

Result: Amal 10, Amal 11, Bilel 12. Chaima and booking 13 are **missing**.

LEFT JOIN: **all rows of the LEFT table**, plus the matches (NULL if there's none).

```
SELECT Customers.FullName, Bookings.BookingID
  FROM Customers LEFT JOIN Bookings ON Customers.CustomerID = Bookings.CustomerID;
```

Result: Amal 10, Amal 11, Bilel 12, **Chaima NULL**. ☞ "**including customers who haven't booked**" means a LEFT JOIN with Customers on the left.

RIGHT JOIN: **all rows of the RIGHT table**, plus the matches.

```
SELECT Customers.FullName, Bookings.BookingID
  FROM Customers RIGHT JOIN Bookings ON Customers.CustomerID = Bookings.CustomerID;
```

Result: Amal 10, Amal 11, Bilel 12, **NULL 13**. ☐ `A LEFT JOIN B` gives the same result as `B RIGHT JOIN A`: swap the tables.

FULL OUTER JOIN (= FULL JOIN): all rows from both sides. Result: the 3 matches + **Chaima/NULL + NULL/13**. (MySQL doesn't run it: use `LEFT JOIN … UNION … RIGHT JOIN` instead.)

SELF JOIN: a table joined **with itself**, using 2 aliases.

```
SELECT a.FullName, b.FullName FROM Customers a, Customers b
  WHERE a.City = b.City AND a.CustomerID < b.CustomerID;
```

Without the JOIN keyword (same result as INNER):

```
SELECT c.FullName, b.BookingID FROM Customers AS c, Bookings AS b
  WHERE c.CustomerID = b.CustomerID;
```

3 tables:

```
... FROM A INNER JOIN B ON A.id = B.a_id INNER JOIN C ON B.id = C.b_id;
```

Course: MySQL supports 4 types: **INNER, LEFT, RIGHT, SELF**.

B3. UNION

UNION combines the results of 2 SELECTs. They need the **same number of columns**, similar types, and the same column order.

```
SELECT City FROM Customers UNION SELECT City FROM Suppliers;   -- no duplicates
  SELECT City FROM Customers UNION ALL SELECT City FROM Suppliers ORDER BY City;  -- ke
  eps duplicates
```

B4. Aggregates, GROUP BY, HAVING

Aggregates: `COUNT()`, `SUM()`, `AVG()`, `MAX()`, `MIN()` **GROUP BY** puts rows with the same value into one group, to compute an aggregate **per group**.

Clause order (memorize it):

```
SELECT    col, AGG(col)
  FROM      table
  WHERE     condition on ROWS        -- before grouping, no COUNT/SUM here
  GROUP BY  col
  HAVING    condition on GROUPS      -- after grouping, uses COUNT/SUM...
  ORDER BY  col;
```

```
-- number of bookings per date (Lab 8)
  SELECT BookingDate, COUNT(BookingID) FROM Bookings GROUP BY BookingDate;
```

Result: 06-01 → 2, 06-05 → 1, 06-09 → 1.

```
-- the same, but only between two dates (WHERE, before grouping)
  SELECT BookingDate, COUNT(BookingID) FROM Bookings
  WHERE BookingDate BETWEEN '2026-06-01' AND '2026-06-05'
  GROUP BY BookingDate;
```

```
-- total guests per customer, only totals > 4 (HAVING)
  SELECT CustomerID, SUM(Guests) FROM Bookings
  GROUP BY CustomerID HAVING SUM(Guests) > 4;
```

Result: 1 → 6, NULL → 5. (The NULLs form their own group.)

```
-- course example: count per country, more than 4, sorted
  SELECT COUNT(CustomerID), Country FROM Customers
  GROUP BY Country HAVING COUNT(CustomerID) > 4
  ORDER BY COUNT(CustomerID) DESC;
```

```
-- GROUP BY + LEFT JOIN: customers with 0 bookings appear too
  SELECT Customers.FullName, COUNT(Bookings.BookingID) AS NbBookings
  FROM Customers LEFT JOIN Bookings ON Customers.CustomerID = Bookings.CustomerID
  GROUP BY Customers.FullName;
```

Result: Amal 2, Bilel 1, Chaima **0**. (`COUNT(column)` doesn't count NULLs; `COUNT(*)` would give Chaima 1.)

```
-- course example: HAVING + INNER JOIN
  SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders
  FROM Orders INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
  GROUP BY LastName HAVING COUNT(Orders.OrderID) > 5;
```

Rules (knowledge check):

- The statement begins with SELECT.
- The GROUP BY columns are also in the SELECT.
- An aggregate is followed by the column in parentheses.
- GROUP BY comes **after** WHERE, never before.

"Per X", "for each X" or "by X" means `GROUP BY X`.

B5. Subqueries

A subquery is a query inside another query (the **inner** or child query inside the **outer** or parent query). It's always **between parentheses**. It can go in **SELECT, FROM or WHERE** (most often WHERE), and also in INSERT, UPDATE and DELETE.

5 types:

- **Single row**: returns 0 or 1 row.
- **Multiple row**: returns 1 or more rows.
- **Multiple column**: returns 1 or more columns.
- **Correlated**: uses a column of the outer query.
- **Nested**: a subquery inside another subquery.

```
SELECT * FROM Products WHERE Price > (SELECT AVG(Price) FROM Products);   -- compared w
  ith 1 value

  -- EXISTS: true if the subquery returns at least 1 row
  SELECT SupplierName FROM Suppliers
  WHERE EXISTS (SELECT ProductName FROM Products
                WHERE Products.SupplierID = Suppliers.SupplierID AND Price = 20);

  -- ANY (= SOME): true for AT LEAST ONE value of the list
  SELECT ProductName FROM Products
  WHERE ProductID = ANY (SELECT ProductID FROM OrderDetails WHERE Quantity > 100);

  -- ALL: true for EVERY value of the list
  SELECT ProductName FROM Products
  WHERE ProductID = ALL (SELECT ProductID FROM OrderDetails WHERE Quantity = 10);

  -- in FROM: a derived (temporary) table, which MUST have an alias
  SELECT MAX(items), MIN(items), FLOOR(AVG(items))
  FROM (SELECT orderNumber, COUNT(orderNumber) AS items
        FROM orderdetails GROUP BY orderNumber) AS lineitems;
```

B6. INSERT / UPDATE / DELETE / REPLACE (changing the DATA)

```
INSERT INTO Customers (CustomerID, FullName) VALUES (4, 'Dhia');
  INSERT INTO Customers (CustomerID, FullName) VALUES (5,'Eya'), (6,'Fedi');   -- sever
  al rows

  UPDATE Employees SET Salary = Salary * 1.10 WHERE Dept = 'Sales';            -- +10%
  UPDATE Customers SET FullName = 'Amel', City = 'Tunis' WHERE CustomerID = 1;

  DELETE FROM Customers WHERE CustomerID = 3;
```

⚠ **UPDATE or DELETE without WHERE affects EVERY row.**
REPLACE INTO: if the key already exists, the old row is **deleted and replaced** (works like an update). If not, the row is **inserted**.

```
REPLACE INTO Customers (CustomerID, FullName) VALUES (1, 'Amal B.');   -- syntax 1, lik
  e INSERT
  REPLACE INTO Customers SET CustomerID = 7, FullName = 'Ghada';         -- syntax 2, l
  ike UPDATE
```

With subqueries (course):

```
INSERT INTO agent1 SELECT * FROM agents
    WHERE agent_code = ANY (SELECT agent_code FROM customer WHERE cust_country = 'UK');
  UPDATE agent1 SET commission = commission + .02
    WHERE 2 >= (SELECT COUNT(cust_code) FROM customer WHERE customer.agent_code = agent
  1.agent_code);
  DELETE FROM customer1
    WHERE agent_code = ANY (SELECT agent_code FROM agents WHERE working_area = 'Londo
  n');
```

Copying between tables: `INSERT INTO new SELECT … FROM old …`. The **FROM clause** names the existing table.

B7. MySQL constraints (in CREATE TABLE)

Constraints are rules that keep data **valid, accurate, consistent and reliable**. If an insert or update breaks a rule, it's **aborted**.

```
CREATE TABLE Students (
    StudentID INT AUTO_INCREMENT,
    Email VARCHAR(255) UNIQUE,                  -- no duplicates (university email)
    Name VARCHAR(100) NOT NULL,                 -- mandatory
    Age INT CHECK (Age >= 18),                  -- a rule (driving license ≥ 18)
    Rating INT CHECK (Rating BETWEEN 3 AND 10),-- a valid range (domain)
    City VARCHAR(50) DEFAULT 'Tunis',           -- automatic value
    DeptID INT,
    PRIMARY KEY (StudentID),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
  );
  -- composite key: PRIMARY KEY (StudentID, CourseID)
  -- named:         CONSTRAINT PK_Patient PRIMARY KEY (PatientID, SlotID)
```

3 types:

- **Key**: every table has a PK, which is unique, not null and prevents duplicate records.
- **Domain**: allowed values for a column.
- **Referential integrity**: the FK value must exist as a PK (or unique key) in the referenced table.

B8. ALTER TABLE (changing the STRUCTURE)

Action	Syntax
Add a column	`ALTER TABLE Employees ADD Email VARCHAR(255);`
Change the type	`ALTER TABLE Employees MODIFY COLUMN Email VARCHAR(100);`
Rename a column	`ALTER TABLE Employees CHANGE Email BusinessEmail VARCHAR(50);`
Delete a column	`ALTER TABLE Employees DROP COLUMN Email;`
Rename the table	`ALTER TABLE OrderStatus RENAME OrderDeliveryStatus;`
Add a FK	`ALTER TABLE Vehicle ADD FOREIGN KEY (OwnerID) REFERENCES Owner(OwnerID);`

- **MODIFY** = new type, same name. **CHANGE** = new name, and you must rewrite the type.
- **UPDATE** changes the **values**. **ALTER** changes the **columns**.
- **DELETE FROM** removes rows. **DROP COLUMN** removes a column. **DROP TABLE** removes the whole table.

B9. Views

A **view** is a **virtual table**. It **stores no data**; it's an interface to the data in the base tables.

```
CREATE VIEW OrdersView AS SELECT OrderID, Quantity, Cost FROM Orders;
  SELECT * FROM OrdersView;                              -- use it like a table
  UPDATE OrdersView SET Cost = 250 WHERE OrderID = 2;    -- changes the BASE table Orde
  rs
```

3 benefits:

1. **Simplifying access** to data: you can create a view over a join so you don't retype it.
2. **Manipulating data**: filter or update the base table through the view.
3. **Security**: show only the needed columns and hide sensitive ones, such as ClientID and ProductID.

SELF-TEST (cover the answers)

1. Degree vs cardinality?
2. The 3 integrity constraints?
3. Where does the FK go in a 1:N relationship?
4. How do you implement an M:N relationship?
5. Partial vs transitive dependency?
6. "All employees, including those without a department": which join?
7. "Departments with more than 3 employees": WHERE or HAVING?
8. Is 50 included in `BETWEEN 10 AND 50`?
9. Rename the column `tel` to `phone`: MODIFY or CHANGE?
10. What happens with `UPDATE Employees SET Salary = 0;`?
11. Does a view take storage space?
12. LIKE pattern for "second letter is o"?
13. "Customer username is always 9 characters": which type?
14. A name that appears twice in the data: can it be a candidate key?
15. `REPLACE INTO` when the key already exists?

▶ Answers

CLASSIC DS TRAPS

- `= NULL` ☐ → `IS NULL` ☐
- A condition on COUNT or SUM goes in **HAVING**, not WHERE.

- GROUP BY comes **after** WHERE.
- Text needs quotes; numbers don't.
- `%` is a remainder, not a percent.
- UPDATE or DELETE **always with WHERE**, unless you really mean every row.
- CHANGE needs the type again; MODIFY keeps the name.
- A derived table (a subquery in FROM) **must have an alias**.
- A FK can only reference a table that **already exists**, so create the parent tables first.
- M:N is **never** kept as-is: use a junction table.
- Normalization order: **1NF → 2NF → 3NF**. Each form requires the previous one.